

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ПАРАЛЕЛНА РАМКА ЗА МЕТАЕВРИСТИЧНА
ОПТИМИЗАЦИЯ НА КОМБИНАТОРНИ
ЗАДАЧИ: ПРОЕКТИРАНЕ, ПРОГРАМНА
РЕАЛИЗАЦИЯ И ЕКСПЕРИМЕНТАЛНА
ОЦЕНКА НА ПРОИЗВОДИТЕЛНОСТТА**

КУРСОВ ПРОЕКТ

по дисциплина „Метаевристика“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Милена Лазарова, Доц. д-р Аделина Алексиева, Доц. д-р Георги Запрянов

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. ТЕОРЕТИЧНА ОСНОВА, ПРОЕКТИРАНЕ И РЕАЛИЗАЦИЯ	2
1.1. Метаевристики и паралелни схеми	2
1.2. Проектни решения	3
1.3. Архитектура, договори и реализация	3
II. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ОЦЕНКА	5
2.1. Среда и еталонни инстанции	5
2.2. Бюджет, повторения и заплахи за валидността	5
III. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	7
3.1. Цена на оценяването и избор на съседен ход	7
3.2. Качество на четирите метаевристики при равен бюджет	8
3.3. Сравнение между паралелните схеми	9
3.4. Ускорение и ефективност спрямо броя нишки	9
3.5. Фалшиво споделяне и разпределение на времето в една итерация	11
ЗАКЛЮЧЕНИЕ	12
ЛИТЕРАТУРА	13

УВОД

Практически важните задачи за комбинаторна оптимизация са изчислително трудни, тоест точното им решаване е приложимо само за малки входове. Вместо това инженерната практика използва *метаевристики*: общи стратегии за търсене, които се отказват от гаранцията за оптималност в замяна на контрол върху изразходваното време [1]. Формата им, много слабо свързани опити вместо една дълга зависима верига, ги прави удобни за паралелно изпълнение.

Лесното паралелизиране обаче е подвеждащо. Класификацията в [2] показва, че отделните схеми, независим многократен старт, споделяне на общо най-добро решение и островен модел, се различават не само по постигнатото ускорение, а и по това дали решават същата задача: кооперативните схеми променят траекторията на търсенето, тоест променят и качеството на решението. Оттук и проблемът на проекта. Твърдението „реализацията е паралелна и затова е по-бърза“ не е проверимо, докато не бъдат фиксирани едновременно наборът еталонни инстанции, критерият за спиране и начинът за сравняване на качеството. Без тези три условия измереното ускорение може да идва от по-слаба работа на нишка, а не от по-добро използване на машината.

Целта е да се проектира и реализира паралелна рамка за метаевристична оптимизация, в която задачата, метаевристиката и схемата за паралелно изпълнение стоят зад общ договор и се менят независимо, и да се направи количествена оценка на производителността и на качеството при контролирани условия. Задачите са четири: обосновка на проектните решения; реализация на четири метаевристики и три паралелни схеми с внимание към цената на синхронизацията; методика за измерване, която фиксира инстанциите, критерия за спиране, броя повторения и обобщаването; и измерване на ускорението, на ефективността, на качеството и на разпределението на времето.

Обхватът е рамката, метаевристиките в нея и експерименталната оценка. Извън обхвата остават разпределеното изпълнение върху няколко машини, изпълнението върху графични ускорители и точните методи, използвани само като източник на известни оптимални стойности.

Проектът не предлага нова метаевристика и не твърди подобрене спрямо публикувани резултати. Той предлага условия, при които сравнението между вече известни методи е проверимо, и показва с измерване докъде тези условия стигат.

I. ТЕОРЕТИЧНА ОСНОВА, ПРОЕКТИРАНЕ И РЕАЛИЗАЦИЯ

1.1. Метаевристики и паралелни схеми

Задачата за комбинаторна оптимизация се задава с крайно множество допустими решения и целева функция върху него, но множеството расте толкова бързо с размера на входа, че изчерпателното му обхождане е неприложимо. Метаевристиките управляват подчинена евристика, която сама би заседнала в локален оптимум, и се различават по това как балансират изследването и задълбочаването [1, 3]. В проекта влизат четири от тях, покриващи четири различни отношения към споделеното състояние. *Симулираното закаляване* [4] приема влошаващи ходове с вероятност, намаляваща с температурата, тоест зависи изцяло от схемата на охлаждане и от съседството. *Търсенето с забрани* [5] пази краткосрочна памет за скорошните ходове, а забраненият списък е структура с честа проверка за принадлежност, тоест точка, в която разположението на данните влияе на времето за итерация. *Генетичният алгоритъм* [6] оценява индивидите на популацията независимо, тоест се разпределя естествено между нишки. *Мравчената колония* [7] строи решения над обща матрица от следи, тоест споделеното състояние тук е част от алгоритъма.

Паралелните метаевристики се класифицират по това какво точно се разпределя [2, 8]. Независимият многократен старт не изисква комуникация и мащабира почти линейно, но не използва натрупаното от другите нишки; кооперативните схеми го използват, но плащат със синхронизация и с риск от преждевременна сходимост; островният модел стои между двете и въвежда честотата на обmena като допълнителен параметър. Сравнимостта с публикувани резултати изисква инстанции с известни стойности, каквито са TSPLIB [9] и OR-Library [10], а тълкуването на измерена производителност изисква модел, отделящ ограничението от изчисленията от ограничението от паметта; моделът на покривната линия [11] дава такъв, но изисква хардуерни броячи, каквито тук не са снети.

Източниците описват стратегиите и схемите поотделно и всеки фиксира по едно сравнение. Липсва общата рамка, в която задача, стратегия и схема за изпълнение се менят независимо при едни и същи условия, а без нея всяко сравнение между две публикувани реализации е и сравнение между два набора инстанции и два критерия за спиране.

1.2. Проектни решения

Четири независими решения и основанията им са в Табл. 1 (Табл. 1). Динамичният полиморфизъм е избран срещу първоначалното намерение за шаблони по горещия път, защото оценяването заема под 11 на сто от една итерация, §3.5.

Таблица 1. Проектни решения и основанията за всяко от тях

Решение	Алтернативи	Избрано	Основание
Език и среда	управлявана среда; език без стандартен паралелизъм	C++20 [12]	Контрол върху разположението на данните, без събирач на боклук в горещия цикъл.
Паралелизъм	задачно ориентирана среда; директиви над цикли	явни нишки	Предметът на измерването е цената на синхронизацията и тя не бива да е скрита зад чужд планировчик. Задачната среда остава неизмерена.
Договор за задача	статичен полиморфизъм с шаблони	динамичен	Оценяването е под 11 на сто от итерацията, §3.5.
Обмен между нишките	само една от трите схеми	и трите	Образуват редица с нарастваща комуникация; кое се изплаща е проверено в §3.3, а не допуснато.

1.3. Архитектура, договори и реализация

Изискването е едно: задачата, метаверстиката и схемата за паралелно изпълнение трябва да се менят независимо, иначе сравнението между схемите става и сравнение между две реализации на алгоритъма. Оттук следват четири слоя със зависимости само надолу: договор за задача (evaluate, delta, apply, random_move, construct), метаверстики (step, best, inject), схеми за паралелно изпълнение и измервателна рамка. Слойта на алгоритмите не знае колко нишки има, а слойта на задачите не знае кой алгоритъм го вика, тоест добавянето на трета задача не наложи промяна в нито един алгоритъм. И трите задачи използват едно представяне, вектор от цели числа, а кандидатът носи кеширано състояние, тоест общото тегло при раницата и обратната пермутация при търговския пътник.

Метаверстиката е обект със състояние, изпълняващ стъпки, защото нишка трябва

да може да продължи от решение, дошло отвън. Една стъпка обаче означава различно нещо при различните алгоритми, тоест бюджетът се задава във време или в брой оценявания, никога в стъпки, а периодът на обмен се брои в оценявания: период, разумен за отгряването с милиони итерации в секунда, означава нула обмени за мравчената колония с около сто и петдесет, и първата версия, броила итерации, наистина не обменяше нищо. Трите схеми се различават само по обмена: при многократния старт обмен няма и той е еталонът за цената на комуникацията; при кооперативната нишките обновяват обща най-добра стойност; при островния подпопулациите обменят по пръстен, а не по пълен граф, където схемата би се изродила в кооперативната.

Реализацията е около 4300 реда C++20 без нито една задължителна външна зависимост (Табл. 2) и е публична на <https://github.com/Alex-Tsvetanov/anneal>. Липсата на зависимости е условие за възпроизводимост: изграждане, което изисква мениджър на пакети, трето лице няма да повтори. Затова изпълнителят на тестове е сто реда вместо външна рамка, а генераторът е xoshiro256++, защото стандартните разпределения не са специфицирани до бит. Три решения са взети по измерване, а не по очакване, и числата им са в Раздел III: генераторът на съседен ход, чиято първа версия правеше търсенето безполезно; изборът между матрица на разстоянията и координати; и подравняването на данните на всяка нишка на 64 байта. Тестовите са 28 и покриват четири твърдения: че инкременталното оценяване дава същия резултат като пълното и при трите задачи, че всяко върнато решение е допустимо, че при фиксиран бюджет в оценявания резултатът е побитово възпроизводим, и че инстанции с обявен оптимум наистина го имат, защото без такъв свидетел оптимумът е само твърдение.

Таблица 2. Модули на реализацията, брой редове от `wc -l`

Файл	Съдържание	Редове
include/anneal/	договорите Problem и Metaheuristic, xoshiro256++, статистики, TSPLIB	880
src/algorithms.cpp	четирите метаевристики	445
src/parallel.cpp	трите схеми и споделеното състояние	320
src/tsp.cpp, knapsack, coloring	трите задачи и генераторите на инстанции	883
benchmarks/, apps/, src/experiment.cpp	измервателна програма с осем подкоманди, демонстрация, рамка	1010
tests/	изпълнител на тестове и 28 теста	807

II. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ОЦЕНКА

Методиката фиксира предварително какво се измерва и как се обобщава, за да могат разликите да бъдат отдадени на конкретна причина, а не на разлика в условията. Експериментите отговарят на пет въпроса: как се променя времето до фиксиран праг с броя нишки; дава ли кооперативната схема по-добро качество при равен бюджет и каква е цената на обмена; как се разпределя времето между оценяване, съсед и синхронизация; как се подреждат четирите метаевристики; и помага ли векторизацията.

2.1. Среда и еталонни инстанции

Всички измервания са на една машина (Табл. 3) със стойности, прочетени от нея, не от документация. Динамичното управление на честотата не е изключвано, защото средата не го позволява надеждно, и то е една от причините за размаха.

Таблица 3. Конфигурация на машината, на която са направени измерванията

Компонент	Стойност
Процесор	AMD Ryzen 5 3600, 6 физически ядра, 12 логически, 3950 MHz
Система и компилатор	Windows 10.0.26200; g++ 15.2.0 MinGW-w64; -std=c++20 -O3 -DNDEBUG, без -march=native
Изграждане и часовник	CMake 4.3.2, Ninja 1.13.2; std::chrono::steady_clock

Проектът не изтегля инстанции: измерване, което изисква мрежов достъп, не се повтаря от трето лице години по-късно. Инстанциите се генерират, а обявената стойност на всяка е или доказана в затворен вид, или пресметната точно (Табл. 4). Има четец на TSPLIB [9] за EUC_2D, но никое измерване не зависи от него.

2.2. Бюджет, повторения и заплахи за валидността

Използват се два режима на спиране и всеки резултат посочва в кой е получен. При фиксирано време се измерва качеството за еднакъв бюджет стенно време и този режим сравнява алгоритми и схеми. При фиксирано качество се измерва времето до зададен праг, и той единствен отговаря на въпроса за ускорението, защото при фиксирано време повече нишки просто извършват повече работа. За праг се взема *най-лошото*

Таблица 4. Използвани инстанции и произход на обявената стойност

Инстанция	Размер	Стойност	Откъде идва стойността
uniform-1000	1000 града	неизвестна	Случайни точки. Класът връща NaN, тоест качеството се докладва като дължина, а не като отклонение.
grid-24x24	576 града	5760	Решетка със стъпка 10: никое ребро не е по-късо от стъпката, а тестът построява маршрут с дължина точно n пъти стъпката.
knapsack-400	400 предмета	15771	Точен оптимум с динамично оптимизиране, проверен срещу изчерпателно изброяване при 16 предмета.
coloring-150-k8	150 върха, 8 цвята	0 конфликта	Засадена правилна подялба, тоест нулево оцветяване съществува по построение и тестът го предявява.

качество от пилотна серия от пет изпълнения с една нишка по 400 ms, така че тя да го достига при всяко повторение; праг, който тя не достига, би дал безкрайно ускорение. Бюджет в брой оценявания се използва в тестовете като единствения режим с побитова възпроизводимост.

Всяка конфигурация се изпълнява 11 пъти; при пет повторения размахът на времето до праг беше съизмерим с медианата. Резултатът се дава с медиана и междуквартилен размах, защото разпределението на времената е силно изкривено надясно. Квартилите са с линейна интерполация между съседните рангове, тоест определението по подразбиране на NumPy и на R, посочено заради разминаването между трите разпространени определения при малки извадки. Относителното отклонение е $(\text{намерено} - \text{известно}) / \max(1, |\text{известно}|)$.

Профайлър не беше изпълнен и такъв не се твърди: perf го няма на тази система, а четене на часовника около всяка операция би струвало повече от нея. Използваният метод е *аблация*: измерва се цикъл с първата част от итерацията, после с първите две, и разликата е цената на добавената част. Методът не отчита взаимодействието между частите, тоест за кешови пропуски не става. Известните ограничения са пет: една машина; сравнение между конкретни настройки, не между методите; схемите с комуникация правят по-малко оценявания; неизключено управление на честотата; и генерирани инстанции, тоест изводите важат за измереното.

III. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Всяка таблица посочва командата, която я е произвела, тоест клетка може да бъде проследена до изпълнение, а числата са от машината в Табл. 3. Възпроизводимостта при фиксирано начално число е проверена от тест за побитово съвпадение; при бюджет във време такава няма и не се твърди.

3.1. Цена на оценяването и избор на съседен ход

Таблица 5. Цена на едно оценяване: раница, с и без разклонение (вляво); дължина на маршрута, през матрица и от координати (вдясно). Команда: `anneal_bench objective`

Форма	n	ns/оц.	ns/елем.	Източник	n	ns/оц.	ns/град
без	256	187,3	0,732	матрица	200	150,2	0,751
разклонение							
с разклонение	256	218,4	0,853	координати	200	460,4	2,302
без	1024	775,4	0,757	матрица	1000	788,6	0,789
разклонение							
с разклонение	1024	1053,4	1,029	координати	1000	2275,8	2,276
без	4096	3131,8	0,765	матрица	2000	2135,5	1,068
разклонение							
с разклонение	4096	4749,4	1,160	координати	2000	4541,8	2,271
				матрица	4000	31768,5	7,942
				координати	4000	9156,5	2,289

Формата без разклонение е между 1,17 и 1,52 пъти по-бърза (Табл. 5), но обхватът на резултата е ограничен, защото горещият път не прави пълно оценяване. При дължината на маршрута измерването опроверга очакването: матрицата е по-бърза само докато се събира в кеша, а при 4000 града тя е 128 MB, стойността ѝ скача на 7,942 ns на град и координатната форма става 3,5 пъти по-бърза. Реализацията избира по този измерен праг.

Таблица 6. Генератор на ходове при два милиона предложения, `uniform-1000`. Команда: `anneal_bench neighbourhood`

Генератор	Обхват	Дължина	Подобрения	ns на ход
начало, най-близък съсед	няма	29307,2	няма	няма
равномерен 2-opt	999	25086,8	3396	52,1
равномерен 2-opt	50	27443,9	12857	53,9
списък съседи	50	27579,4	63330	83,5
списък съседи	200	26874,9	62803	79,4
списък съседи	999	24964,5	66761	58,3

Всички редове в Табл. 6 тръгват от един и същ маршрут и изпълняват два милиона предложения при чисто низходящо търсене, тоест без метаевристика над генератора. Ограничаването на обхвата до петдесет позиции влошава резултата от 24964,5 на 27579,4, тоест с 10,5 на сто, защото забранява далечните поправки. Списъкът от най-близки съседи променя крайния резултат малко, 24964,5 срещу 25086,8, но свива разпределението на влошаващите ходове, чиято медиана пада от 355 на 82, а отгряването калибрира началната си температура точно от него: при равномерно теглене веригата тръгва около четири пъти по-гореца от нужното и не подобрява началния си маршрут. Тези стойности описват отпаднала конфигурация и не се възпроизвеждат от команда в хранилището, затова стоят като **[TODO: стойности от отпадналата конфигурация, ако бъдат премерени наново]**. Качеството при фиксиран бюджет се определя от съседството поне толкова, колкото от стратегията.

3.2. Качество на четирите метаевристики при равен бюджет

Таблица 7. Качество при бюджет 250 ms, една нишка, 11 повторения. Команда: anneal_bench quality

Инстанция	Медиана				МКР			
	отгр.	табу	ген.	мравч.	отгр.	табу	ген.	мравч.
uniform-1000	25481,11	24488,29	27011,09	27336,82	141,36	181,21	342,06	197,26
grid-24x24	5851,13	5760,00	5760,00	5760,00	29,71	8,28	10,32	21,54
knapsack-400	15724,00	15527,00	15753,00	15707,00	7,00	26,50	19,00	10,00
coloring-150-k8	0,00	0,00	4,00	11,00	0,00	0,00	2,00	4,50
Отклонение от известната стойност					Оценявания			
uniform-1000	неизв.	неизв.	неизв.	неизв.	1 941 903	4 341 681	52 058	1249
grid-24x24	1,582 %	0,000 %	0,000 %	0,000 %	2 404 751	4 280 241	84 400	3617
knapsack-400	0,298 %	1,547 %	0,114 %	0,406 %	3 875 215	5 078 961	105 956	145
coloring-150-k8	0 конфл.	0 конфл.	4 конфл.	11 конфл.	1 196 431	747 441	157 642	2081

Подредбата зависи от задачата и това е основното наблюдение (Табл. 7). Табу търсенето е най-добро при двете инстанции на търговския пътник и най-лошо при раницата, където изостава с 1,55 на сто от точния оптимум срещу 0,114 на сто за генетичния алгоритъм; отгряването и табу търсенето решават оцветяването точно, а популационните методи не. Твърдение от вида „метод X е по-добър от метод Y“ не се поддържа от тези измервания. Броят оценявания се различава с четири порядъка, защото едно преминаване на колонията строи шестнадесет пълни решения; затова бюджетът е във време, а обменът се брой в оценявания.

3.3. Сравнение между паралелните схеми

Таблица 8. Трите схеми при 8 нишки, 300 ms, 11 повторения, табу търсене. Команда: anneal_bench schemes

Инстанция	Схема	Медиана	МКР	Синхр. ms	CAS	Приети
uniform-1000	многократен	24320,92	116,99	0,000	0	0
uniform-1000	кооперативна	24095,71	137,22	23,372	8	1614
uniform-1000	островна	24136,65	144,82	12,733	0	808
knapsack-400	многократен	15552,00	20,00	0,000	0	0
knapsack-400	кооперативна	15591,00	53,00	0,253	0	54
knapsack-400	островна	15608,00	24,50	15,024	0	47
coloring-150-k8	многократен	0,00	0,00	0,000	0	0
coloring-150-k8	кооперативна	0,00	0,00	0,045	0	38
coloring-150-k8	островна	0,00	0,00	2,184	0	33

Комуникацията се изплаща, но малко (Табл. 8). При търговския пътник кооперативната схема подобрява медианата с 0,93 на сто спрямо многократния старт, а островната с 0,76; при раницата подредбата се обръща, с 0,25 и 0,36 на сто. Цената е 23,4 ms сумарно за осемте работника при бюджет 300 ms всеки, тоест около 1 на сто от процесорното време, а неуспешните атомарни обновявания са 8 при над 1600 приемания, тоест състезанието е практически нулево. Островната схема плаща повече в синхронизация при раницата (15,0 ms) от кооперативната (0,253 ms), но при кооперативната се брои само времето вътре в критичната секция, тоест двете колони не са сравними. Това е ограничение на инструментиранието, а не резултат.

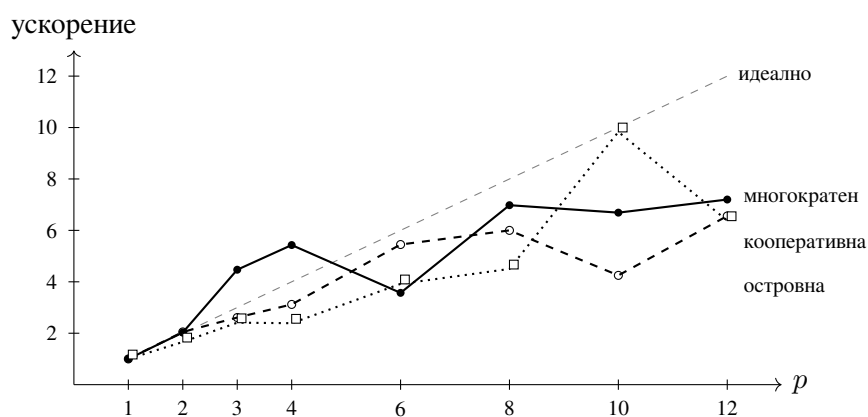
3.4. Ускорение и ефективност спрямо броя нишки

Прагът е най-лошото качество от пилотна серия от пет изпълнения по 400 ms с една нишка, тоест 24730,9 при най-добро пилотно 24283,1. Всяка клетка е медиана от 11 изпълнения с горна граница 3000 ms и всички достигат прага (Табл. 9).

Кривите се отклоняват от идеалната в две посоки (Фиг. 1). При малък брой нишки ускорението надхвърля идеалното, 4,47 при три и 5,43 при четири за многократния старт, но обяснението е в съседната колона: размахът при една нишка е 84,22 ms при медиана 87,45 ms, тоест разпределението е силно изкривено надясно, а минимумът от p тегления при тежка опашка пада по-бързо от $1/p$. Свръхлинейността е свойство на разпределението, не на машината, и изчезва при дванадесет нишки, където размахът е 9,73 ms. Над четири

Таблица 9. Време до праг 24730,9 на uniform-1000, табу търсене, 11 повторения. Команда: anneal_bench speedup

Нишки	Многократен старт				Кооперативна				Островна			
	ms	МКР	Уск.	Еф.	ms	МКР	Уск.	Еф.	ms	МКР	Уск.	Еф.
1	87,45	84,22	1,00	1,00	61,24	58,63	1,00	1,00	60,48	58,87	1,00	1,00
2	43,12	45,08	2,03	1,01	29,81	34,61	2,05	1,03	36,50	51,52	1,66	0,83
3	19,57	28,50	4,47	1,49	23,49	14,85	2,61	0,87	25,13	20,31	2,41	0,80
4	16,10	30,05	5,43	1,36	19,62	9,39	3,12	0,78	25,31	14,48	2,39	0,60
6	24,50	28,26	3,57	0,59	11,25	15,42	5,45	0,91	15,42	10,82	3,92	0,65
8	12,53	35,29	6,98	0,87	10,22	8,92	6,00	0,75	13,45	26,96	4,50	0,56
10	13,08	25,89	6,69	0,67	14,40	21,10	4,25	0,43	6,15	31,11	9,83	0,98
12	12,14	9,73	7,20	0,60	9,33	13,27	6,56	0,55	9,48	6,21	6,38	0,53



Фигура 1. Ускорение спрямо брой нишки за трите схеми, с нанесена идеална линия. Данните са от Табл. 9

нишки кривите се сплескват и ефективността пада до 0,53 до 0,60 при шест физически и дванадесет логически ядра, а синхронизацията не обяснява това, защото е около 1 на сто от процесорното време. Измерванията обаче не установяват дали причината е споделянето на изпълнителни единици или пропускателната способност на паметта, тъй като разграничението изисква хардуерни броячи, каквито не бяха снети. Отклоненията от тенденцията, например 3,57 при шест нишки, са в границите на разсейването.

3.5. Фалшиво споделяне и разпределение на времето в една итерация

Таблица 10. Фалшиво споделяне при двадесет милиона увеличения на нишка (вляво) и аблация на една итерация на отгряването (вдясно). Команди: `anneal_bench falsesharing` и `profile`

Нишки	обща, ms	отделни, ms	отн.	Тяло на цикъла	uniform-200		uniform-1000	
					ns	дял	ns	дял
2	114,3	8,4	13,53	само случайно теглене	1,9	3,2 %	1,8	2,9 %
4	267,2	24,0	11,14	плюс генериране на ход	45,4	73,2 %	44,3	70,2 %
8	534,8	34,9	15,35	плюс инкрементално	50,6	8,7 %	50,5	10,3 %
12	519,1	41,8	12,41	оценяване				
				плюс приемане и	59,4	14,8 %	60,6	16,6 %
				прилагане пълно преоценяване, за машаб	147,4	2×	788,8	13×

Подравняването на състоянието на всеки работник на кеш линия подлежеше на проверка (Табл. 10). Ефектът е от 11 до 15 пъти и не намалява с броя нишки, тоест подравняването се оправдава, но експериментът измерва най-лошия случай, запис при всяка итерация.

Разпределението на времето е по метода на аблацията от §2.2 и противоречи на очакването, с което проектирането беше правено. Очакваше се оценяването на целевата функция да доминира; то заема между 8,7 и 10,3 на сто, а доминира генерирането на съседен ход, с между 70 и 73 на сто, защото прави няколко непредсказуеми обръщания към паметта. Последният ред дава машаб: при 1000 града пълно преоценяване на всяка итерация би струвало 13 пъти повече от цялата итерация. Оттук следва и защо векторизацията има ограничен обхват: тя ускорява оценяване, каквото горещият път не прави. Това е отрицателен резултат.

ЗАКЛЮЧЕНИЕ

Проектирана и реализирана е паралелна рамка за метаевристична оптимизация с три независими точки на разширение: четири метаевристики, три паралелни схеми, три задачи, без външни зависимости и с 28 автоматични теста.

Измерванията дават четири резултата. Времето до фиксиран праг качество намалява до 7,20 пъти при дванадесет нишки, а при две до четири нишки ускорението е свръхлинейно, до 5,43; обяснението не е в машината, а в разпределението, чийто размах при една нишка е 84,22 ms при медиана 87,45 ms. Кооперативната схема подобрява медианното качество с 0,93 на сто при търговския пътник и го влошава при раницата, тоест обменът не е универсално подобрене, а цената му е около 1 на сто от процесорното време. Подредбата на четирите метаевристики зависи от задачата. И генерирането на съседен ход заема около 70 на сто от една итерация, докато оценяването на целевата функция е под 11, тоест очакването, върху което беше правено проектирането, е опровергано.

Предимствата се формулират спрямо измереното. Разделянето на договорите работи: добавянето на трета задача не наложи промяна в нито един алгоритъм. Единната мярка за бюджет в оценявания позволява алгоритми, чиито итерации се различават с четири порядъка по цена, да обменят решения смислено. И инкременталното оценяване се изплаща: при 1000 града пълното преоценяване би струвало 13 пъти повече от цялата итерация.

Недостатъците са три и всеки е показан от измерване, а не от преглед на кода. Качеството зависи от съседството поне толкова, колкото от стратегията: едно и също табу търсене стига до 27579 или до 24965 според това дали обхватът на хода е ограничен. Броячът за време в синхронизация мери различни неща при кооперативната и при островната схема, тоест колоните не са сравними, което е дефект на инструментиранието. И единадесет повторения не разграничават съседни конфигурации, когато размахът е съизмерим с медианата.

Продължението следва от отворения въпрос в §3.4: разграничаването между споделяне на изпълнителни единици и ограничение по памет изисква хардуерни броячи. Следват разпределеното изпълнение върху няколко машини, което поставя обмена при по-висока цена на комуникацията, и автоматичната настройка на параметрите. Задачно ориентираната среда остава разгледана, но неизмерена алтернатива.

ЛИТЕРАТУРА

- [1] C. Blum and A. Roli, “Metaheuristics in combinatorial optimization: Overview and conceptual comparison,” *ACM Computing Surveys*, vol. 35, no. 3, pp. 268–308, 2003.
- [2] T. G. Crainic and M. Toulouse, “Parallel meta-heuristics,” in *Handbook of Metaheuristics* (M. Gendreau and J.-Y. Potvin, eds.), pp. 497–541, Boston, MA: Springer, 2 ed., 2010.
- [3] E.-G. Talbi, *Metaheuristics: From Design to Implementation*. Hoboken, NJ: John Wiley & Sons, 2009.
- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [5] F. Glover, “Future paths for integer programming and links to artificial intelligence,” *Computers & Operations Research*, vol. 13, no. 5, pp. 533–549, 1986.
- [6] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.
- [7] M. Dorigo, V. Maniezzo, and A. Colorni, “Ant system: Optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.
- [8] E. Alba, ed., *Parallel Metaheuristics: A New Class of Algorithms*. Hoboken, NJ: John Wiley & Sons, 2005.
- [9] G. Reinelt, “TSPLIB — a traveling salesman problem library,” *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.
- [10] J. E. Beasley, “OR-Library: Distributing test problems by electronic mail,” *Journal of the Operational Research Society*, vol. 41, no. 11, pp. 1069–1072, 1990.
- [11] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [12] International Organization for Standardization, Geneva, *ISO/IEC 14882:2020 — Programming languages — C++*, 2020.